PRINTXYZ

A-to-Z Production-Level Website & Business Blueprint

A detailed brief for the AI coding agent building Printxyz: storefront, custom-order operations, payments, billing, logistics, security, marketing, analytics, and scale.

• Read this document and inspect the existing repository before changing code.

• Do not blindly rebuild the project. Report the current stack, architecture, existing features, gaps, risks, and recommended implementation order.

• Preserve working functionality unless a change is necessary and approved.

• Separate production-ready features from mockups and placeholders. Never claim a feature works until it is implemented and tested.

• Never invent payment, shipping, tax, legal, courier, or provider API behaviour. Verify current official documentation.

• Build in small phases with database migrations, tests, error handling, logs, and rollback plans.

• Ask for missing business decisions rather than silently inventing policies.

• Keep credentials on the server. Never expose secrets in frontend code or browser storage.

Printxyz should be a real ecommerce operation, not just a storefront. The system must support:

• Product discovery and customer acquisition.

• Personalised product configuration and photo uploads.

• Clear pricing, payment collection, order status, and customer communication.

• Print production, quality control, packing, courier booking, tracking, and returns.

• Business expenses, payment reconciliation, invoice records, and profitability.

• Secure customer/admin access, privacy, backups, monitoring, and recovery.

• Growth without needing to rewrite the whole platform.

The owner currently has a Bambu Lab A1 and black and ivory-cream filament, without paint. Design product offerings around realistic printer capability and actual materials. Do not promise colours, finishes, sizes, details, or turnaround times that cannot reliably be delivered.

Important financial distinction: ■1 lakh monthly revenue is not ■1 lakh monthly profit. Track both. Do not promise the target can be achieved through website software alone.

Customer

• Browse products and categories.

• Search, filter, compare, and view product details.

• Configure available options and upload permitted custom images.

• Save cart and checkout details securely.

• Pay, receive invoice/receipt, track orders, request eligible support, and manage profile.

• Request account/data deletion where applicable.

Owner/admin

• Manage products, prices, stock, custom orders, production queue, customers, orders, refunds, shipments, coupons, content, reports, and settings.

• Access business data only after strong authentication and server-side authorisation.

• Review and approve sensitive actions such as refunds, manual payment marking, cancellation after production starts, and shipment cancellation.

Staff (future-ready, optional)

• Limited roles such as production, fulfilment, support, and catalog management.

• Least-privilege permissions. Staff should not automatically see payment secrets, all customer data, or admin settings.

Global layout

• Responsive header with logo, categories, search, account, cart, and mobile navigation.

• Clear announcement strip only when useful.

• Consistent footer with contact details, policies, support links, social links, and business information.

• Brand palette: navy `#0F172A`, orange `#F97316`, white `#FFFFFF`, soft grey `#F8FAFC`, muted text `#64748B`, success `#16A34A`, error `#DC2626`.

• Accessible typography, clear focus states, readable contrast, consistent spacing and responsive design.

• Loading, empty, error, offline, success, and disabled states for relevant UI.

• No fake reviews, fake scarcity, fabricated ratings, or misleading countdowns.

Homepage

• Clear value proposition explaining what Printxyz sells and why it is different.

• Primary CTA to shop and a separate CTA for personalised/custom gifts.

• Categories, bestsellers, new products, gift occasions, and selected products.

• Real product photos and examples of customisation.

• Trust information: payment methods, production process, delivery estimates, support, and policies.

• Customer reviews only when genuine and collected appropriately.

• FAQ, contact/support access, and useful footer.

• Optimised mobile layout for social-media traffic.

Catalogue/category pages

• Product cards with image, name, price, available options, and genuine badges.

• Search, sort, and useful filters.

• Pagination or efficient incremental loading.

• Empty search and no-results state.

• Canonical URLs and indexability rules.

• Filters should not generate unlimited duplicate SEO pages.

Product detail page

• Multiple product images, zoom or gallery, clear description, dimensions, material, colour, care guidance, limitations, estimated production/dispatch time, shipping/return terms, price and available options.

• Variant availability and stock status.

• Customisation form where relevant.

• Delivery/pincode check only if supported by a reliable serviceability source.

• Add to cart and buy now with clear validation and feedback.

• Related products only if relevant.

• Do not claim a 3D preview exists unless it really works.

Search and discovery

• Search by product name, category, use, and relevant keywords.

• Typo tolerance only if practical.

• Useful suggestions and no-result alternatives.

• Track searches with privacy-conscious analytics.

Product data

Each product should support:

• Stable product ID, slug, name, description, category, status.

• Images and alt text.

• Product type: standard, made-to-order, personalised, or digital/design-only if ever offered.

• Variants/options and per-combination SKU where needed.

• Price, compare-at price only when legitimate, cost estimate, tax configuration, weight and packed dimensions.

• Stock or production capacity, estimated print time, material use, listing state.

• SEO title, meta description, canonical URL, and social sharing image.

• Product-specific customisation schema and eligibility rules.

Price calculation

The server must calculate and validate all totals. Never trust client-supplied price, discount, shipping, or tax values.

A configurable pricing model may account for:

• Filament/material cost.

• Estimated printer time and electricity.

• Labour, design/setup, and post-processing.

• Packaging.

• Expected failed prints/remakes.

• Payment processing fees.

• Shipping and insurance only where actually purchased/available.

• Applicable tax treatment.

• Advertising/customer acquisition cost and desired margin.

Display the customer's final price and fees clearly before payment. Define whether options are per item or one-time; prevent add-ons and discounts from accidentally replacing base variant prices.

Inventory and capacity

• Distinguish physical stock from made-to-order capacity.

• Prevent overselling with atomic stock/reservation logic.

• Define when stock is reserved and when reservations expire.

• Track filament by colour/material and optional batch.

• Track print queue load and promised dispatch windows.

• Avoid promising instant dispatch for items that must be printed.

Recommended state sequence:

• Customer selects custom product and options.

• Customer uploads image(s) and provides required instructions.

• Server validates file size, type, dimensions and allowed content; strip unnecessary metadata where appropriate.

• Store files privately or behind short-lived signed access; do not expose customer uploads through public permanent URLs.

• Order is created with clear payment/advance terms.

• Owner reviews image feasibility and sends preview/mockup if the business offers one.

• Customer approves or requests a permitted revision.

• Track revision count and approval timestamp.

• Production begins only after required approval/payment.

10. Quality check, packing, shipment, tracking and completion.

11. Apply a defined photo retention/deletion schedule.

Required safeguards:

• Clear upload consent and privacy explanation.

• Supported formats and maximum size shown before upload.

• File validation on the server, not only in the browser.

• Rate limits and abuse protection.

• No access to another customer's upload by guessing IDs.

• No public indexing of uploaded photos.

• Documented deletion and backup-retention policy.

• Clear cancellation cutoff, revision policy, preview approval, remake, refund, and balance-payment rules.

Do not invent a policy when one is not decided. Surface those decisions to the owner.

• Persistent cart with server-side price revalidation.

• Correct variant/customisation and quantity handling.

• Guest checkout if compatible with the chosen design; account creation should not be an unnecessary blocker.

• Address form with Indian state, PIN code, phone validation, and clear error messages.

• Shipping price and delivery estimate only from configured rules/provider data.

• Coupon eligibility, expiry, usage limits, minimum basket, product exclusions, and concurrency checks.

• Clear order summary, total, payment schedule, refund/cancellation policy and consent to relevant terms.

• Prevent duplicate order creation on repeated taps or network retries.

• Preserve cart on recoverable errors without exposing private data.

• Support accessible keyboard and mobile interaction.

COD and custom orders

• COD is optional, not mandatory. Decide eligibility based on risk, shipping provider support, product type, and business policy.

• Custom products may require an advance and may not be eligible for COD; disclose terms before checkout.

• Track COD amount, delivery outcome, remittance/settlement, RTO charges and reconciliation separately.

• Do not allow mixed-cart rules to become ambiguous.

• Use a supported payment gateway checkout and official server-side verification.

• Verify signatures and webhooks according to current provider documentation.

• Treat webhooks as potentially duplicated or out of order; implement idempotency.

• Browser redirect/success page is not proof of payment.

• Store gateway order/payment IDs, amount, currency, status, timestamps and reconciliation state.

• Separate order status, payment status, production status and shipment status.

• Handle pending, failed, partially paid, paid, refund pending, partially refunded, and refunded states as applicable.

• Reconcile gateway transactions against internal orders and settlement reports.

• Never store card details or payment secrets yourself.

• Refunds must use the provider's supported process and be auditable.

• Log who approved a manual payment correction or refund.

• Avoid recording full sensitive payment payloads in logs.

Billing/invoices

• Generate a stable invoice/receipt reference with customer, seller, line items, discounts, shipping, tax treatment, totals, payment references, and issue date.

• Prevent duplicate invoice numbers and accidental edits to issued documents.

• Keep credit notes/refund records when required.

• Tax documents must reflect the business's actual registration status and applicable law. Do not label a document a GST tax invoice unless legally appropriate.

• Make tax settings configurable; have a qualified Indian accountant confirm registration, invoicing, tax rate, place-of-supply, and interstate selling obligations before launch.

• Export records for bookkeeping and reconciliation.

• Keep invoice PDFs access-controlled.

Use a shipping aggregator/provider such as Shiprocket only after confirming account eligibility, supported API functions, rates, and serviceability from its current official documentation.

One-click shipping workflow

• Customer places order; no shipment is booked yet.

• Payment/approval rules are checked.

• Owner prints the item, performs QC, packs it and confirms real packed weight/dimensions.

• Admin clicks Ready to Ship.

• Backend validates order, address, parcel measurements, payment/COD policy and eligibility.

• Backend creates shipment using provider API and stores provider IDs.

• Request courier selection/assignment and label where supported.

• Schedule pickup where supported; show status/errors.

• Customer receives tracking link after booking is confirmed.

10. Tracking events update shipment status safely.

11. Delivery exceptions, returns and claims are recorded.

Shipping requirements

• Keep API credentials server-side.

• Prevent duplicate shipments through idempotency and database constraints.

• Validate weight, dimensions, serviceability and addresses.

• Support label retrieval/printing and pickup status where provider supports it.

• Verify webhook signatures where supported; safely process duplicate/out-of-order events.

• Have safe retries, backoff, error messages, audit history and manual fallback.

• Keep shipment state separate from payment and order state.

• Record shipping quotes and actual charges where available.

• Do not promise insurance/compensation unless confirmed for the shipment.

• Do not automatically refund or replace a damaged parcel without a defined review process.

• Store courier, AWB/tracking ID, tracking URL, pickup/delivery events, return status and claim status.

The owner dashboard should have:

• New orders, paid orders, pending approvals, printing queue, QC queue, ready-to-ship queue, shipments in transit, exceptions, delivered, cancellations and refunds.

• Filters by date, status, product, payment method, courier, and custom/standard order.

• Search by order ID, customer, phone/email where appropriate, and tracking number.

• Order timeline showing important events and who/what caused them.

• Internal notes separated from customer-visible messages.

• Print-job checklist: file/version, material/colour, quantity, estimated time, QC result and remake reason.

- Customer support action and shipment exception action.
- Bulk actions only where safe and reversible.
- CSV export with permission checks and privacy safeguards.
- Publish clear shipping, cancellation, customisation, returns, refunds, damaged-on-arrival and warranty/defect policies.
- Policies must comply with applicable Indian consumer law; have them reviewed before launch.
- Create a support ticket or case linked to an order.
- Capture issue type, customer evidence, timestamps, owner response, resolution and cost.
- Define evidence and deadlines for courier claims.
- Distinguish manufacturing defect, transit damage, customer-approved custom design, wrong item and change-of-mind cases.
- Provide manual approval for refund, remake, replacement or rejection.
- Send customer updates and maintain an audit trail.
- Do not use hidden or misleading policy wording.
- Secure login and logout; session expiry and revocation.
- Profile and address management.
- Order list, order details, invoice access and tracking.
- Custom-order approval history and upload access.
- Support requests and notification preferences.
- Password/login recovery if password auth exists.
- Transactional email/SMS/WhatsApp only when configured and permitted.
- Use approved templates and required consent for business messaging channels.
- Retry failed notifications without duplicates.
- Separate transactional updates from marketing messages.
- Never expose one customer's data to another customer.
- Strong admin authentication and server-side role-based access control.
- Do not rely on hiding admin links in the frontend.
- Store secrets in deployment environment/secret manager.
- Rate-limit login, upload, coupon, checkout and other abuse-prone endpoints.
- Validate and sanitise input; use safe database queries and output escaping.
- Apply secure session/cookie settings appropriate to the architecture.
- Protect state-changing routes against CSRF where relevant.
- Configure CORS narrowly; use HTTPS.
- Prevent IDOR: every order, upload, invoice and account resource must check ownership/permission.
- Use least privilege and log sensitive admin actions.
- Keep dependencies updated and remove unused packages.
- Do not reveal stack traces, credentials, or internal details to customers.
- Add security headers and safe error handling.
- Never commit `.env`, API keys, tokens, private keys, customer uploads, or production database dumps to Git.
- Publish a clear privacy policy and explain why data is collected.
- Collect only information needed for order fulfilment, support, legal compliance and consented analytics.
- Explain custom photo use, access, retention and deletion.
- Define retention periods for photos, invoices, orders, logs and backups with professional/legal input where required.
- Provide a reasonable data access/deletion request process subject to legal recordkeeping duties.
- Use private object storage and expiring links for sensitive uploads.
- Do not send customer photos to third-party AI tools without an appropriate lawful basis, clear disclosure, and safeguards.

• Avoid storing sensitive data in analytics tools.

• Restrict support/admin access and audit access to customer files.

SEO

• Unique titles and descriptions for indexable pages.

• Clean product/category URLs, canonical links and sitemap.

• Robots rules for private/account/checkout/admin pages.

• Correct product structured data only for accurate current price, availability and product information.

• Open Graph/social metadata so product links preview well on messaging and social platforms.

• Optimised images, descriptive alt text, fast page loads and useful content.

• Noindex private or duplicate pages as appropriate.

• Avoid fake reviews and unsupported claims.

Advertising and analytics

• Configure analytics and Meta Pixel/Conversions API only with correct consent/privacy configuration where required.

• Track meaningful events: product view, search, add to cart, checkout start, payment attempt, verified purchase, refund and lead/custom-order enquiry.

• Deduplicate browser/server events using supported event IDs.

• Purchase value should reflect the defined business event: do not report full order value as collected revenue when only an advance was received without clearly defining and reconciling the metric.

• Exclude test orders and prevent duplicate purchase events.

• Track campaign, product, conversion rate, customer acquisition cost, average order value, refunds and contribution margin.

• Do not optimise only for clicks or vanity metrics.

Conversion improvements

• Clear product value, dimensions, materials, delivery timing, customisation process and policies.

• Real product photos and demonstrations.

• Transparent pricing and shipping.

• Trustworthy support details.

• Fast mobile experience.

• Simple checkout.

• Genuine reviews and user-submitted product photos with permission.

• Abandoned-cart reminders only where appropriate, configured and compliant.

Show date filters and definitions for:

• Gross order value.

• Net sales after discounts and refunds.

• Cash actually collected.

• Gateway fees and settlements.

• Shipping collected versus actual shipping cost.

• Material, packaging and estimated labour costs.

• Advertising spend and attributed orders.

• Contribution margin per order/product.

• Gross profit and estimated net profit, with assumptions stated.

• Average order value, conversion rate, repeat purchase rate, refund/defect rate, on-time dispatch and delivery.

• Best/worst products by margin, not just sales.

• Production hours and capacity utilisation.

• COD remittance, failed delivery and return-to-origin costs.

Never label revenue as profit. Show whether a figure is actual, estimated, or incomplete due to missing expense data.

Automations may include:

• Order confirmation after order creation.

• Payment confirmation only after verified payment.

• Custom preview approval request.

• Production status updates where useful.

• Shipment booked and tracking link after confirmed booking.

• Delivery updates from verified tracking events.

• Admin alerts for failed payments, upload failures, shipment errors, low stock, and unresolved support cases.

• Daily summary of orders, pending actions and exceptions.

• Retry queues for transient API/notification failures.

Every automation needs idempotency, retry rules, logging, and a way to see whether it succeeded. Do not send duplicate or false notifications.

• Responsive, mobile-first layouts.

• Optimised image sizes and lazy loading below the fold.

• Avoid loading heavy scripts unnecessarily.

• Pagination or efficient queries for large product/order lists.

• Database indexes for frequently queried fields.

• Timeouts and error handling for external APIs.

• Safe caching: never cache private invoices, account pages, or customer uploads publicly.

• Accessible labels, keyboard navigation, focus states and semantic HTML.

• Meaningful loading, empty, offline and error states.

• Monitor slow requests, failed jobs, API errors and server crashes.

• Test on common mobile screen sizes and low/unstable network conditions.

• Separate development, staging/test and production configuration.

• Never use real payment or courier credentials in public test environments.

• Automated database backups with restricted access and documented retention.

• Back up required uploads and configuration safely.

• Test restoration, not just backup creation.

• Monitor uptime, API failures, database connectivity, disk/storage use and job queue health.

• Use structured logs with request IDs while minimising personal data.

• Alert the owner on critical failures.

• Document deployment, environment variables, migration, rollback and recovery steps.

• Use a staging/test workflow before major production changes.

• Keep a deployment checklist and known-issues list.

Do not force a new stack if the current project is working. Inspect it first. For a typical custom-built system, a maintainable architecture may include:

• Frontend: existing React-based application and existing styling approach.

• Backend: existing Node.js/Express API if that is what the project already uses.

• Database: existing MongoDB setup with schema validation and indexes.

• Image storage: private/signed Cloudinary or object-storage workflow as appropriate.

• Payments: existing gateway integration with verified server-side events.

• Shipping: provider API behind a backend service/module.

• Notifications: existing email/WhatsApp/SMS provider behind an abstraction.

• Hosting: current deployment platform, with secrets and environment-specific configuration.

• Error monitoring and analytics selected based on budget and privacy needs.

This is a possible structure, not a mandate. Do not introduce a second database, framework, or hosting platform without explaining why it is needed.

Before launch, test at least:

• Successful and failed payment flows.

• Duplicate payment webhooks and delayed events.

• Price tampering and invalid coupons.

• Concurrent stock/checkout attempts.

• Customer access to only their own orders, uploads and invoices.

• Admin route and role enforcement.

• Invalid/oversized uploads and private image access.

• Custom-order approval, revisions, cancellation cutoff and balance payment.

• Shipment creation, duplicate prevention, label retrieval, pickup errors, tracking retries and manual fallback.

• Refund and partial refund handling.

• COD collection/settlement where enabled.

• Mobile navigation, checkout and form validation.

• Email/WhatsApp notification failure and retry.

• Backup restore and production rollback.

• SEO metadata, sitemap, social previews and analytics deduplication.

• Accessibility and performance basics.

Do not launch with critical payment, privacy, authorisation, or order-integrity failures.

Phase 0 — Audit and decisions

• Inspect repository and deployment.

• Inventory current features and tests.

• Identify missing business decisions: cancellation cutoff, revisions, payment schedule, shipping/COD, tax treatment, retention, refund policy and service areas.

• Produce a risk list and implementation plan.

Phase 1 — Reliable core commerce

• Product/catalogue management.

• Product pages, cart, checkout and server-side price validation.

• Secure authentication and customer order history.

• Payment verification and order state model.

• Basic invoices/receipts appropriate to business status.

• Essential policies and customer contact.

Phase 2 — Custom orders and fulfilment

• Private uploads and custom-order workflow.

• Preview approval and revision limits.

• Production queue and QC checklist.

• Shipping-provider integration, labels, pickup and tracking.

• Customer transactional notifications.

Phase 3 — Operations and financial control

• Refund/support cases.

• Expenses and payment reconciliation.

• Inventory/material tracking and print capacity.

• Admin roles and audit logs.

• Daily operational reports.

Phase 4 — Growth systems

• SEO, product structured data and social previews.

• Analytics/Meta event deduplication.

• Coupon management and campaign attribution.

• Customer retention and consent-based messaging.

• Product profitability and capacity reports.

Phase 5 — Scale and hardening

• Monitoring, backup/restore drills and recovery documentation.

• Load and concurrency tests appropriate to traffic.

• Queue-based retries for background tasks where needed.

• Staff permissions and operational workflows.

• Ongoing security and performance reviews.

Treat each phase as a set of tested milestones, not a single giant coding task.

The website cannot guarantee income. Use unit economics to understand what must be true.

Example scenarios, for illustration only:

Average order value	Contribution per order after variable costs	Orders needed for ■1,00,000 contribution/month
■1,000	■250	400
■1,500	■500	200
■2,000	■800	125
■2,500	■1,000	100

These figures are arithmetic examples, not forecasts. Contribution per order must deduct relevant variable costs such as material, failed prints, labour allowance, packaging, shipping subsidy, payment fees, discounts, refunds/remakes and advertising. Fixed expenses and taxes may still need to be deducted to reach net profit.

For a single Bambu Lab A1, validate print hours and labour capacity before targeting high order counts. Prioritise products with healthy contribution margin, repeatable production, low damage rates, and clear customer demand. Track real data weekly and adjust product pricing and marketing based on evidence.

Create a decision list and ask the owner to confirm:

• Initial delivery area: Patna only or all India.

• Product catalogue and product-specific production times.

• Custom advance percentage and balance-payment timing.

• Preview and revision limits.

• Cancellation cutoff after approval/production begins.

• COD eligibility.

• Shipping pricing: included, flat rate, or calculated.

• Returns/refunds/remakes and damaged shipment policy.

• GST/tax status and invoice configuration, confirmed with a qualified professional.

• Photo retention/deletion duration.

• Notification channels and customer consent.

• Staff accounts/roles now or later.

• Advertising and analytics consent configuration.

• Which integrations already have active accounts and credentials.

A feature is not complete just because its screen looks correct. It is production-ready only when:

• Its server-side rules exist.

• Data is stored and validated correctly.

• Permissions and privacy are enforced.

• Success, failure, retry, and duplicate-event paths are handled.

• The UI shows real state from the backend.

• Relevant automated/manual tests pass.

• Logs and recovery behaviour are adequate.

• Required provider setup is documented.

• It has been tested in a safe environment before real customers depend on it.

Final instruction to the AI coding agent

Start with an audit, not a rewrite. Compare the current project with this blueprint and return:

• Existing stack and architecture.

• Feature-by-feature status: complete, partial, missing, or unsafe.

• Critical blockers before launch.

• Missing owner decisions.

• Phased implementation plan with files/modules to change.

• Tests needed for each phase.

Then implement in small, reviewable phases. Do not claim production readiness, legal compliance, verified payments, live shipment booking, or successful notifications unless each has been implemented and verified.